

FuncMage

Master the Ancient Arts of Functional Programming

Summary: Welcome to FuncMage Chronicles! In the year 2142, you are a Function Mage learning the ancient arts of higher-order functions, decorators, and lambda spells. Master these powerful techniques to restore balance to the digital realm.

Version: 3.2

Contents

I Foreword **2**

II AI Instructions **3**

III Introduction **5**

IV Common Instructions **7**
IV.1 Python Requirements 7
IV.2 Authorized Imports 7
IV.3 Forbidden 8
IV.4 Output Specifications 8

V Exercise 0: Lambda Sanctum **9**

VI Exercise 1: Higher Realm **12**

VII Exercise 2: Memory Depths **15**

VIII Exercise 3: Ancient Library **18**

IX Exercise 4: Master's Tower **21**

X Submission and Peer-Review **24**

Chapter I

Foreword

Welcome, aspiring Function Mage, to the year 2142!

In this cyberpunk future, the digital realm faces chaos. Ancient programming techniques have been forgotten, and only the **Function Mages** possess the knowledge to restore balance. You are one of these chosen few, regardless of your background, identity, or experience level—every mage brings unique strengths to our diverse guild.

The **Lambda Codex**, a legendary artifact containing the secrets of **higher-order functions**, **decorators**, and **lexical scoping**, has been scattered across the digital dimensions. As a Function Mage, you must master these ancient arts to reunite the fragments and save both the virtual and physical worlds.

Your journey will take you through five mystical realms, each teaching you a fundamental aspect of functional programming. You'll meet fellow mages from all walks of life—Alex the Lambda Specialist, Jordan the Decorator Master, Riley the Scope Guardian, and many others who will guide you on this adventure.

Remember, young mage: **functions are first-class citizens** in Python, meaning they can be passed around, stored in variables, and transformed just like any other data. This power, when wielded correctly, can create elegant, reusable, and powerful spells that adapt to any situation.

Your quest begins now. The digital realm awaits your unique contribution to the Function Mage legacy!

Chapter II

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter III

Introduction

Welcome to FuncMage Chronicles:

You've mastered Python's fundamentals, conquered object-oriented programming, and learned to handle exceptions with grace. Now it's time to discover the most elegant and powerful programming paradigms: **the techniques of functional programming**!

In this epic adventure, you'll explore five mystical realms:

- **Exercise 0**: Lambda Sanctum - Master anonymous functions and lambda expressions
- **Exercise 1**: Higher Realm - Discover the power of higher-order functions
- **Exercise 2**: Memory Depths - Understand lexical scoping and closures
- **Exercise 3**: Ancient Library - Explore the functools module's treasures
- **Exercise 4**: Master's Tower - Create powerful decorators and class methods

Each realm presents challenges that will expand your understanding of Python's functional capabilities.



PREREQUISITES: This activity requires solid mastery of Python functions, classes, exception handling, and basic data structures. You should be comfortable with Python syntax and object-oriented concepts before embarking on this functional programming journey.



Focus on understanding **why** functional programming patterns are powerful, not just **how** to implement them. A great Function Mage understands the philosophy behind the magic.



HELPER TOOL AVAILABLE: This activity includes a helper tool to assist your learning journey! Check the attachments, extract the archive and look for:

- `data_generator.py` - It will help you generate realistic test data for all exercises with mages, artifacts, and spells.

Use `python3 data_generator.py` to generate test data for your implementations, and to test during review.

Chapter IV

Common Instructions

IV.1 Python Requirements

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- Exception handling should protect the data streams from corruption.
- Use **type hints** for all function signatures and return types.
- Focus on demonstrating functional programming patterns clearly.
- Each exercise should be in its own file with the specified name.
- Write clean, readable code that demonstrates your understanding.

IV.2 Authorized Imports

- **typing** module - For advanced type hints
- When using **Callable** as a type hint, use **collections.abc**
- **itertools** module - For advanced iteration patterns
- Essential modules for this project **functools** and **operator** will be specified if allowed in each exercise.
- All standard classes and collections are authorized, along with their methods (`int`, `str`, `list`, `dict`, etc.).
- All built-in functions are authorized, but `eval()` and `exec()`.
- Standard library modules as needed for the exercises

IV.3 Forbidden

- External libraries (no pip install)
- File I/O operations (focus on in-memory processing)
- Complex algorithms (keep focus on functional patterns)
- Using eval() or exec()
- Global variables (embrace functional purity when possible)

IV.4 Output Specifications

Each exercise provides terminal examples showing the suggested output format. While the core structure must be maintained, you may customize messages to reflect your understanding of the file operations, as long as the essential information is preserved.

Chapter V

Exercise 0: Lambda Sanctum

	Exercise0
ex0	
Directory: *ex0/*	
Files to Submit: `lambda_spells.py`	
Authorized: `map, filter, sorted, min, max, round, sum, len`	



For this exercise, you must use lambda expressions for all transformations. Do not use the 'def' keyword to create named functions for simple operations, like one-time tasks or easy to read operations. The goal is to master anonymous functions and functional programming patterns.

The Lambda Sanctum: Mastering Anonymous Functions

Welcome to the Lambda Sanctum, young mage! Here, the ancient art of **anonymous functions** is taught. Lambda expressions are like quick incantations—short, powerful spells that can transform data without the ceremony of full function definitions.



Your Mission: Master the art of lambda expressions by helping the Sanctum's guardian, Sage Lambda, organize magical artifacts using anonymous functions. Learn to create functions on-the-fly for data transformation.

The Challenge

Create a file `lambda_spells.py` that contains functions demonstrating lambda mastery:
Function Signatures:

```
def artifact_sorter(artifacts: list[dict]) -> list[dict]
def power_filter(mages: list[dict], min_power: int) -> list[dict]
def spell_transformer(spells: list[str]) -> list[str]
def mage_stats(mages: list[dict]) -> dict
```

Implementation Requirements

artifact_sorter(artifacts) - Sort magical artifacts:

- Use `sorted()` with a lambda to sort by 'power' level (descending)
- Each artifact is a dict: `{'name': str, 'power': int, 'type': str}`
- Return the sorted list

power_filter(mages, min_power) - Filter mages by power:

- Use `filter()` with a lambda to find mages with power >= min_power
- Each mage is a dict: `{'name': str, 'power': int, 'element': str}`
- Return a list of filtered mages

spell_transformer(spells) - Transform spell names:

- Use `map()` with a lambda to add "* " prefix and " *" suffix
- Input: list of spell names (strings)
- Return a list of transformed spell names

mage_stats(mages) - Calculate statistics:

- Use lambdas with `max()`, `min()` to find:
- Most powerful mage's power level
- Least powerful mage's power level
- Average power level (rounded to 2 decimals)
- Return dict: `{'max_power': int, 'min_power': int, 'avg_power': float}`

Expected Output Example:

```
$> python3 lambda_spells.py

Testing artifact sorter...
Fire Staff (92 power) comes before Crystal Orb (85 power)

Testing spell transformer...
* fireball * * heal * * shield *
```



How do **lambda expressions** make code more concise? When should you use lambda vs. regular function definitions?

Chapter VI

Exercise 1: Higher Realm

	Exercise1
ex1	
Directory: *ex*1/	
Files to Submit: `higher_magic.py`	
Authorized: `callable(), Callable`	

The Higher Realm: Functions Operating on Functions

Welcome to the Higher Realm, where functions become the subjects of other functions! Here, you'll learn that functions are **first-class citizens**—they can be passed as arguments, returned from other functions, and stored in data structures.



Your Mission: Help the Realm's guardian, Mage Functional, create a spell-crafting system where functions can modify, combine, and enhance other functions. Master the art of higher-order functions!

The Challenge

Create a file `higher_magic.py` that demonstrates higher-order function mastery:

Context

Every spell in your grimoire follows the same contract:

```
def spell(target: str, power: int) -> str
```

A spell takes a target and a power level and returns a description string of what happened.

Examples:

```
def heal(target: str, power: int) -> str:
  return f"Heal restores {target} for {power} HP"
```

If you need inspiration for power values or target, use data_generator.py to get sample data.

The following functions must be implemented: Function Signatures:

```
def spell_combiner(spell1: Callable, spell2: Callable) -> Callable
def power_amplifier(base_spell: Callable, multiplier: int) -> Callable
def conditional_caster(condition: Callable, spell: Callable) -> Callable
def spell_sequence(spells: list[Callable]) -> Callable
```

Implementation Requirements

spell_combiner(spell1, spell2) - Combine two spells:

- Return a new function that calls both spells with the same arguments
- The combined spell should return a tuple of both results
- Example: `combined = spell_combiner(fireball, heal)`

power_amplifier(base_spell, multiplier) - Amplify spell power:

- Returns a function with the same signature as the original spell
- Returns a new spell where the power is multiplied before casting.
- Example: `mega_fireball = power_amplifier(fireball, 3)`

conditional_caster(condition, spell) - Cast spell conditionally:

- Returns a new spell that only casts if a condition is True.
- If condition fails, return "Spell fizzled"
- Both condition and spell receive the same arguments

spell_sequence(spells) - Create spell sequence:

- Return a function that casts all spells in order
- Each spell receives the same arguments
- Returns a list of all spell results

Short Output Example:

```
$> python3 higher_magic.py

Testing spell combiner...
Combined spell result: Fireball hits Dragon, Heals Dragon

Testing power amplifier...
Original: 10, Amplified: 30
```

Your program must demonstrate use and efficiency of all your spells modifiers functions,



How do **higher-order functions** enable code reuse and composition? What makes functions "first-class citizens" in Python?



From which package is it recommended to use Callable? What is the purpose of callable()?

Chapter VII

Exercise 2: Memory Depths

	Exercise2
ex2	
Directory: *ex2/*	
Files to Submit: `scope_mysteries.py`	
Authorized: `nonlocal`	

The Memory Depths: Lexical Scoping and Closures

Deep in the Memory Depths, the ancient secrets of **lexical scoping** are guarded. Here, functions remember the environment where they were created, capturing variables in mystical **closures** that persist beyond their original scope.



Your Mission: Help the Memory Keeper, Sage Closure, understand how functions can "remember" variables from their creation environment. Master closures and lexical scoping to create persistent magical effects!

The Challenge

Create a file `scope_mysteries.py` that demonstrates lexical scoping mastery
Function Signatures:

```
def mage_counter() -> Callable
def spell_accumulator(initial_power: int) -> Callable
def enchantment_factory(enchantment_type: str) -> Callable
def memory_vault() -> dict[str, Callable]
```

Implementation Requirements

mage_counter() - Create a counting closure:

- Return a function that counts how many times it's been called
- Each call should return the current count (starting from 1)
- The counter should persist between calls
- Creating two separate counters must yield independent state.
- Use closure to maintain state without global variables

spell_accumulator(initial_power) - Create power accumulator:

- Return a function that accumulates power over time
- Each call adds the given amount to the total power
- Return the new total power after each addition
- Start with initial_power as the base

enchantment_factory(enchantment_type) - Create enchantment functions:

- Return a function that applies the specified enchantment
- The returned function takes an item name and returns enchanted description
- Format: "enchantment_type item_name" (e.g., "Flaming Sword")
- Each factory creates functions with different enchantment types

memory_vault() - Create a memory management system:

- Return a dict with 'store' and 'recall' functions
- 'store' function: takes (key, value) and stores the memory
- 'recall' function: takes (key) and returns stored value or "Memory not found"
- Use closure to maintain private memory storage

Expected Output Example:

```
$> python3 scope_mysteries.py
Testing mage counter...
counter_a call 1: 1
counter_a call 2: 2
counter_b call 1: 1

Testing spell accumulator...
Base 100, add 20: 120
Base 100, add 30: 150

Testing enchantment factory...
Flaming Sword
Frozen Shield

Testing memory vault...
Store 'secret' = 42
Recall 'secret': 42
Recall 'unknown': Memory not found
```



How do **closures** enable functions to "remember" their creation environment? What are the benefits of lexical scoping in functional programming?



Why is **global** forbidden, but **nonlocal** allowed? What are the key differences?

Chapter VIII

Exercise 3: Ancient Library

	Exercise3
ex3	
Directory: *ex*3/	
Files to Submit: `functools_artifacts.py`	
Authorized: `functools, operator`	

The Ancient Library: Functools Treasures

In the Ancient Library, the most powerful functional programming artifacts are stored. The **functools** module contains legendary tools: `reduce`, `partial`, `wraps`, and more. These artifacts can transform how you approach complex problems.



Your Mission: Help the Library's curator, Archivist Functools, catalog and demonstrate the power of these ancient artifacts. Learn to wield reduce, partial, and other functools treasures!

The Challenge

Create a file `functools_artifacts.py` that demonstrates functools mastery
Function Signatures:

```
def spell_reducer(spells: list[int], operation: str) -> int
def partial_enchanter(base_enchantment: Callable) -> dict[str, Callable]
def memoized_fibonacci(n: int) -> int
def spell_dispatcher() -> Callable[[Any], str]
```

Implementation Requirements

spell_reducer(spells, operation) - Reduce spell powers:

- Use `functools.reduce` to combine all spell powers
- Support operations: "add", "multiply", "max", "min"
- Use `operator` module functions (add, mul, etc.)
- Return the final reduced value
- If spells is empty, return 0
- If operation is unknown, properly handle the error

partial_enchanter(base_enchantment) - Create partial applications:

- Take a base enchantment function with signature (power: int, element: str, target: str) -> str
- Use `functools.partial` to create 3 specialized versions
- Each version pre-filling power=50 and the element

memoized_fibonacci(n) - Cached fibonacci:

- Use `functools.lru_cache` decorator for memoization
- Implement fibonacci sequence calculation
- Function should return the nth Fibonacci number
- The cache should improve performance for repeated calls
- Return the nth fibonacci number



`You can verify caching works via memoized_fibonacci.cache_info().`

spell_dispatcher() - Create single dispatch system:

- Use decorator `functools.singledispatch` to create a spell system
- The base function receives Any and handles unknown spell type
- Handle different types: int (damage spell), str (enchantment), list (multi-cast)
- Return the dispatcher function
- Each type should have appropriate spell behavior

Expected Output Example:

```
$> python3 functools_artifacts.py

Testing spell reducer...
Sum: 100
Product: 240000
Max: 40

Testing memoized fibonacci...
Fib(0): 0
Fib(1): 1
Fib(10): 55
Fib(15): 610

Testing spell dispatcher...
Damage spell: 42 damage
Enchantment: fireball
Multi-cast: 3 spells
Unknown spell type
```



How does **functools.reduce** enable powerful data aggregation? What are the performance benefits of **memoization** with lru_cache?

Chapter IX

Exercise 4: Master's Tower

	Exercise4
ex4	
Directory: *ex4/*	
Files to Submit: `decorator_mastery.py`	
Authorized: `functools.wraps, staticmethod`	

The Master's Tower: Decorator Mastery and Class Methods

At the pinnacle of your journey lies the Master's Tower, where the most advanced Function Mages learn to create **decorators**—magical wrappers that can transform any function's behavior. Here, you'll also master `@staticmethod` and understand how decorators work with classes.



Your Mission: Prove your mastery to the Tower's guardian, Grandmaster Decorator, by creating powerful decorators that can enhance any spell or method. This is your final test as a Function Mage!

The Challenge

Create a file `decorator_mastery.py` that demonstrates decorator mastery: Function Signatures:

```
def spell_timer(func: Callable) -> Callable
def power_validator(min_power: int) -> Callable
def retry_spell(max_attempts: int) -> Callable

class MageGuild:
  @staticmethod
  def validate_mage_name(name: str) -> bool
  def cast_spell(self, spell_name: str, power: int) -> str
```

Implementation Requirements

spell_timer(func) - Time execution decorator:

- Create a decorator that measures function execution time
- Print "Casting function_name..." before execution
- Print "Spell completed in X.XXX seconds" after execution (3 decimal places)
- Use `functools.wraps` to preserve original function metadata
- Return the original function's result

power_validator(min_power) - Parameterized validation decorator:

- Create a decorator factory that validates power levels
- Applied on a standalone function
- If power is valid (>= min_power), execute the function normally
- If invalid, return "Insufficient power for this spell"
- Use `functools.wraps` properly

retry_spell(max_attempts) - Retry decorator:

- Create a decorator that retries failed spells
- If function raises an exception, retry up to max_attempts times
- Print "Spell failed, retrying... (attempt n/max_attempts)"
- If all attempts fail, return "Spell casting failed after max_attempts attempts"
- If one attempt succeeds, return its result normally

MageGuild class - Demonstrate staticmethod:

- `validate_mage_name(name)` - Static method that checks if name is valid
- Name is valid if it's at least 3 characters and contains only letters/spaces
- `cast_spell(self, spell_name, power)` - Instance method
- Should use the power_validator decorator with min_power=10
- When power is valid, return "Successfully cast spell_name with <power> power"
- Otherwise return "Insufficient power for this spell"



You can use time.sleep to simulate waiting time between spells.

Expected Output Example:

```
$> python3 decorator_mastery.py
Testing spell timer...
Casting fireball...
Spell completed in 0.101 seconds
Result: Fireball cast!

Testing retrying spell...
Spell failed, retrying... (attempt 1/3)
Spell failed, retrying... (attempt 2/3)
Spell casting failed after 3 attempts
Waaaaaaagh spelled !

Testing MageGuild...
True
False
Successfully cast Lightning with 15 power
Insufficient power for this spell
```



How do **decorators** enable separation of concerns? What's the difference between @staticmethod and regular instance methods?

Chapter X

Submission and Peer-Review

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the peer-review. Don't hesitate to double-check the names of your files to ensure they are correct.



During peer-review, you may be asked to explain functional programming concepts, demonstrate how closures work, or show how decorators transform functions. Focus on understanding the concepts, not just the implementation.



Keep your implementations focused on demonstrating the functional programming concepts clearly. Avoid over-engineering-the goal is to show mastery of higher-order functions, closures, and decorators.



Congratulations, Function Mage! You've mastered the ancient arts of functional programming. These techniques will make your code more elegant, reusable, and powerful. Use them wisely in your future adventures!